KANN AUDITS

SECURITY REVIEW

FOR RAETO



PREPARED FOR
Raeto

PREPARED BY
Kann Audits

DATE REPORTED
11 November 2026

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	AUDITORS	06
05	METHODOLOGY	07
06	RISK CLASSIFICATION	08
07	FINDINGS SUMMARY	09
08	DETAILED FINDINGS	10
	L-01 A write-off values the claims' part of a failed sink at its last booked price, so claimants share the sink's loss instead of being paid first	10
	L-02 Rounded-down claim cuts at one write-off make a later write-off owe more shares than the vault holds, so the last claimer's part is stuck	17
	L-03 The best-effort withdrawals fall back to `maxWithdraw` / `maxRedeem`, which Morpho VaultV2 always reports as 0, so deleverage and partial recovery take nothing when the sink is short	21
	L-04 The `dustTolerance` waiver is checked per withdrawal, so a holder can split an in-loss exit and leave their gap to the holders who stay	27
	I-01 The stock vault counts empty write-offs toward `MAX_WRITE_OFFS`	31
	I-02 A liquidation that repays every borrow share skips the pre-book	32
	I-03 RaetoVault can re-attach a source that is still pending; writing it off then counts its shares twice	33
	I-04 A detach freezes exits through a queued venue switch, which voids the 2-day exit window	34
09	DISCLAIMER	35
10	ABOUT KANN AUDITS	36

01 - OVERVIEW

AUDIT SUMMARY

Kann Audits performed a focused follow-up security review of the Raeto vaults fix branch from 10 to 11 November 2026. The review examined loss accounting, failed-sink write-offs, partial withdrawals and detachment controls at the audited commit. The eight distinct findings in this report were compared with the final commit. Remediation status follows Raeto's responses to the issues and the merged fix.

TOTAL FINDINGS

8 ISSUES

CRITICAL	–
HIGH	–
MEDIUM	–
LOW	04
INFORMATIONAL	04

ENGAGEMENT DETAILS

PREPARED FOR	Raeto
PREPARED BY	Kann Audits
DATE REPORTED	11 November 2026
REVIEW PERIOD	10 – 11 November 2026
LANGUAGE	Solidity
CHAIN	Robinhood Chain
SCOPE	1,501 nSLOC
TIMELINE	2 days
METHODS	Manual review, reproduction, fix review

02 - WHAT THE PROTOCOL DOES

PROTOCOL OVERVIEW

Raeto puts idle assets on Robinhood Chain to work and turns the interest into AI credits. RaetoVault takes USDG deposits and lends them through a Morpho ERC-4626 vault. Depositors keep their principal; the interest goes to Raeto's treasury and is credited to each depositor as `earned`, which Raeto converts into AI credits off-chain.

Each RaetoStockVault accepts one tokenized stock, with one deployment per stock. The vault posts the stock as collateral in a curated Morpho Blue market, borrows a modest amount of USDG against it (around 30% loan-to-value, never above 45%) and lends that USDG through a Morpho USDG vault. Holders keep their stock, and what the USDG earns minus what the loan costs is credited to them as interest. If Morpho liquidates part of the collateral, the USDG the liquidator repaid becomes a claim of the holders at that time, and while a vault is in loss a holder exits by adding their share of the gap. RaetoMarketPay pays a credit-marketplace order in one transaction; it has no owner and holds no funds.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	Kann-Audits/Raeto-Contracts-Audit
AUDITED COMMIT	be838893ff8d1fc3d47aeb1238d80045bf72f9e8
FINAL COMMIT	0f91b116327c250d7a5b3db942f7383d9b5b78e0

#	FILE	NSLOC
01	raeto-vaults/src/RaetoVault.sol	500
02	raeto-vaults/src/RaetoStockVault.sol	1,001
	Total	**1,501**

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

0X	**0xscater** SECURITY RESEARCHER
KA	**Kann** SECURITY RESEARCHER

05 - HOW WE WORK

METHODOLOGY

This focused review traced the vault accounting and exit paths affected by the earlier remediation, including failure and partial liquidity cases.

The review reproduced the reported behaviour against the audited commit and compared the protocol's fixes and regression tests with the stated root causes. The remediation labels below reflect Raeto's replies.

STEP BY STEP

OUR SECURITY PROCESS

01 **Kickoff & Context**
Identify the audited commit, affected contracts and intended behaviour of each fix.

02 **Scope & Preparation**
Pin the audited and final commits and map each reported issue to its code path.

03 **Fix Branch Review**
Trace the changed vault accounting and exit paths from the audited fix branch.

04 **Expert Security Review**
Review sink failures, claim accounting, withdrawal gaps and privileged detachment paths.

05 **Findings & Proofs**
Record each distinct issue and its proof of concept or reproduction path.

06 **Mitigation Review**
Compare the final commit and Raeto's responses with the reported root causes and regression tests.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- Failed ERC-4626 sinks, write-offs and liquidation-claim priority
- Morpho VaultV2 partial liquidity and zero-valued maxWithdraw/maxRedeem views
- Liquidation pre-booking, empty write-offs and detachment controls
- Rounding across repeated write-offs and in-kind holder claims
- In-loss withdrawal gaps and the dustTolerance waiver
- Protocol responses and regression tests on the final commit

06 - HOW WE RATE ISSUES

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

CRITICAL	HIGH	MEDIUM	LOW	INFO
-	-	-	04	04

TOTAL 08 FIXED 08 ACKNOWLEDGED 00

ID	TITLE	SEVERITY	STATUS
L-01	A write-off values the claims' part of a failed sink at its last booked price, so claimants share the sink's loss instead of being paid first	LOW	FIXED
L-02	Rounded-down claim cuts at one write-off make a later write-off owe more shares than the vault holds, so the last claimer's part is stuck	LOW	FIXED
L-03	The best-effort withdrawals fall back to `maxWithdraw` / `maxRedeem`, which Morpho VaultV2 always reports as 0, so deleverage and partial recovery take nothing when the sink is short	LOW	FIXED
L-04	The `dustTolerance` waiver is checked per withdrawal, so a holder can split an in-loss exit and leave their gap to the holders who stay	LOW	FIXED
I-01	The stock vault counts empty write-offs toward `MAX_WRITE_OFFS`	INFORMATIONAL	FIXED
I-02	A liquidation that repays every borrow share skips the pre-book	INFORMATIONAL	FIXED
I-03	RaetoVault can re-attach a source that is still pending; writing it off then counts its shares twice	INFORMATIONAL	FIXED
I-04	A detach freezes exits through a queued venue switch, which voids the 2-day exit window	INFORMATIONAL	FIXED

08 - ISSUE BY ISSUE

DETAILED FINDINGS

L-01 A write-off values the claims' part of a failed sink at its last booked price, so claimants share the sink's loss instead of being paid first

SEVERITY LOW STATUS FIXED LOCATION RaetoStockVault.sol

SUMMARY

Liquidation claims come first on the vault's USDG (L267). When a sink is written off with claims unpaid, #22 and #25 set aside part of the sink's shares for those claims. They size that part at `sinkRate`, the sink's price recorded at the last `_accrue` before the detach. A sink is written off because it failed, and nothing updates its price while it is detached. If it lost value, that price is too high, so the claimants get too few shares and share the loss pro rata with the holders. The difference goes to the holders, including anyone who deposited after the liquidation.

Our recommendation for #22 suggested this price. It only keeps claims first when the sink has kept its value.

FINDING DESCRIPTION

```
uint256 r = sinkRate[w.token];                                          // L794: price
before the detach
uint256 cs = Math.min(w.shares, Math.mulDiv(need, RATE_UNIT, r, Math.Rounding.Ceil)); // shares for
`need` at that price
uint256 v = Math.min(need, Math.mulDiv(cs, r, RATE_UNIT));
...
_liqPoolUnits = units - covered;                                        // claims cut by
the full booked value
```

The claims are cut by the booked value `v`, but the `cs` shares they receive are worth only `v × (real price / booked price)`. On the redeem path, the same loss falls on the holders first and the claims are paid in full.

ATTACK PATH

1. Alice holds 10 shares, and a liquidation leaves her a 450 USDG claim.
2. Bob deposits 10 stock afterwards. The sink's price is booked at that point.
3. The sink loses half its value, and its views fail. The owner calls `detachSink`.
4. The owner calls `acceptDetachLoss`. The claims' part is sized at the old price, so it is worth half of what alice is owed.
5. Bob, as a holder, keeps the shares that would have paid the rest of her claim.

IMPACT

Measured in the PoC (USDG; the written-off sink is worth half its booked price):

	ALICE (CLAIM 450)	BOB (DEPOSITED AFTER)
Redeem the sink (claims first)	+277.02	−277.02
Write it off	+138.51	−138.51

The claimant loses 138.51 of her claim to a holder who joined after the liquidation. The amount moved is the sink's loss on the claims' part, multiplied by the other holders' share of the vault.

LIKELIHOOD

Low. It needs a write-off with liquidation claims unpaid, and a sink that lost value, which is the usual reason for a write-off. For the loss to move between people, someone must also hold shares but not claims.

PROOF OF CONCEPT

`raeto-vaults/test/report/WriteOffAtStalePriceShiftsClaimLossToClaimant.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {console2} from "forge-std/Test.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {SVLiqBase, MockSink} from "../manual/lib/SVLiqHarness.sol";

/// Liquidation claims come first on the vault's USDG. When a sink is written off, `_setAsideForClaims` sizes the
/// claimants' part at the sink's last booked price (`sinkRate`). A sink is written off because it failed, and its
/// price isn't updated while it is detached, so that price is usually too high: the claimants get too few shares and
/// bear the sink's loss pro rata with the holders, instead of being paid first. The difference goes to the holders,
/// including one who deposited after the liquidation.
/// Harness: the real RaetoStockVault (main at be83889), live Morpho Blue v1.0.0 bytecode (etched), mock USDG / stock /
/// oracle / sink. Run (from raeto-vaults/):
///   FOUNDRY_EVM_VERSION=cancun forge test --match-path test/report/WriteOffAtStalePriceShiftsClaimLossToClaimant.t.sol -vv
contract WriteOffAtStalePriceShiftsClaimLossToClaimantTest is SVLiqBase {
    address alice = makeAddr("alice");
    address bob = makeAddr("bob");

    function _fund(address u) internal {
        usdg.mint(u, 100_000e6);
        vm.prank(u);
        usdg.approve(address(v), type(uint256).max);
    }

    /// Exit with all shares (paying any gap), take the in-kind part, redeem every sink share held: USDG net.
    function _settle(address u, bool writeOff) internal returns (int256) {
        uint256 u0 = usdg.balanceOf(u);
        uint256 sh = v.sharesOf(u);
        vm.startPrank(u);
        v.withdrawTo(sh, u, 0, type(uint256).max);
        try v.claimLiquidation() {} catch {}
        if (writeOff) v.claimWrittenOff(0);
        uint256 b = sink.balanceOf(u);
        if (b > 0) sink.redeem(b, u, u);
        vm.stopPrank();
        return int256(usdg.balanceOf(u)) - int256(u0);
    }

    function test_writeOff_claimantBearsSinkLossThatRedeemWouldGiveToHolders() public {
        _dep(alice, 10e18);
        _lever();
        v.accrue();
        _liquidate(_bs() / 2, 4_500);
        v.accrue();
        uint256 claimA = v.liquidationOwed(alice);
        _dep(bob, 10e18); // bob joins after the liquidation; the sink's price is booked here
        _fund(alice);
        _fund(bob);
```

```
        // the sink loses half its value and stops answering; the owner detaches it
        sink.slash(sink.totalAssets() / 2);
        vm.mockCallRevert(address(sink), abi.encodeWithSelector(MockSink.convertToAssets.selector),
"sink down");
        vm.prank(owner);
        v.detachSink();
        uint256 snap = vm.snapshotState();

        // the sink pays out again (at its real, lower value) and is redeemed: claims are paid first
        vm.clearMockedCalls();
        vm.prank(owner);
        v.redeemRetired(IERC4626(address(sink)));
        int256 aliceRedeem = _settle(alice, false);
        int256 bobRedeem = _settle(bob, false);
        vm.revertToState(snap);

        // written off instead: the claims' part is sized at the old price
        vm.prank(owner);
        v.acceptDetachLoss(IERC4626(address(sink)));
        vm.clearMockedCalls();
        int256 aliceWriteOff = _settle(alice, true);
        int256 bobWriteOff = _settle(bob, true);

        console2.log("alice's claim", claimA);
        console2.log("redeem (claims first): alice / bob");
        console2.logInt(aliceRedeem);
        console2.logInt(bobRedeem);
        console2.log("write-off:             alice / bob");
        console2.logInt(aliceWriteOff);
        console2.logInt(bobWriteOff);

        assertLt(aliceWriteOff, aliceRedeem - 100e6); //@test the claimant bears part of the sink's loss
        assertApproxEqAbs(bobWriteOff - bobRedeem, aliceRedeem - aliceWriteOff, 1e4); //@test the later
depositor gets it
    }
}
```

Output:

```
Ran 1 test for
test/report/WriteOffAtStalePriceShiftsClaimLossToClaimant.t.sol:WriteOffAtStalePriceShiftsClaimLossToCla
imantTest
[PASS] test_writeOff_claimantBearsSinkLossThatRedeemWouldGiveToHolders() (gas: 1932281)
Logs:
  alice's claim 450000000
  redeem (claims first): alice / bob
  277023121
  -277023122
  write-off:             alice / bob
  138511560
  -138511562
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.76ms (1.29ms CPU time)
```

RECOMMENDATION

Don't price the failed sink at all. Keep the claims as USDG claims, and pay them from what the sink actually pays out:

- **At the booking,** don't carve a claims part out of the sink. The claim cut, `sinkRate`, `_simCredit`, `_writeOffClaimUnits` and the claims half of `_inKind` are then no longer needed.
- **Fund the claims from the sink.** Add a permissionless `fundClaimsFromWriteOff(i)` that redeems from the written-off sink only what the unpaid claims still need beyond the vault's other USDG. If the sink is worth less than that, it takes everything the sink pays. It never takes more, so nothing it brings back can become interest.
- **Release the holders afterwards.** Holders take their part in kind once the claims are backed, or once the sink's shares are used up. The first release locks the write-off, so it can't be drawn for claims again.

```
struct WriteOff { address token; uint256 units; uint256 unitsLeft; bool released; uint256 booking; }
mapping(uint256 ⇒ uint256) private _bookingNeedLeft; // set at booking: the claims these write-offs
back, valued then

function fundClaimsFromWriteOff(uint256 i) external nonReentrant returns (uint256 got) {
    if (detachPending() || i ⩾ _writeOffsBooked) revert Paused();
    WriteOff storage w = writeOffs[i];
    if (w.released) return 0;
    _accrue();
    uint256 need = _claimsNeed(); // _liqPoolUnits - _grossUsdg(), or 0
    uint256 budget = _bookingNeedLeft[w.booking];
    if (need == 0) { _bookingNeedLeft[w.booking] = 0; return 0; } // claims backed: later shortfalls
never draw here
    if (need > budget) need = budget;
    if (need == 0) return 0;
    IERC4626 s = IERC4626(w.token);
    uint256 sh0 = s.balanceOf(address(this));
    if (sh0 == 0) return 0;
    uint256 bal0 = usdg.balanceOf(address(this));
    try s.withdraw(need, address(this), address(this)) {}
    catch {
        // worth less than the claims need, or only partly liquid: claims first, so take all it pays now
        try s.redeem(sh0, address(this), address(this)) {}
        catch {
            uint256 m = _payableNow(s); // not maxRedeem: Morpho VaultV2 reports 0 (see L-04)
            if (m > need) m = need;
            if (m > 0) try s.withdraw(m, address(this), address(this)) {} catch {}
        }
    }
    got = usdg.balanceOf(address(this)) - bal0;
    _bookingNeedLeft[w.booking] = got ⩾ budget ? 0 : budget - got;
}

function claimWrittenOff(uint256 i) external nonReentrant returns (uint256 out) {
    if (i ⩾ _writeOffsBooked) revert Paused();
    WriteOff storage w = writeOffs[i];
    uint256 bal = IERC20(w.token).balanceOf(address(this));
    if (!w.released) {
        _accrue();
        if (bal > 0 && _bookingNeedLeft[w.booking] > 0 && _claimsNeed() > 0) revert ClaimsFirst(); //
claims first
        w.released = true;
    }
    ... // holders' part: Math.mulDiv(bal, u, w.unitsLeft); w.unitsLeft -= u (see L-02)
}
```

The budget matters. Without it, a write-off that no holder has claimed yet would back *any* later shortfall. If the vault reopened and a new sink then lost value, the old holders' shares would pay for it. In our test that took all 44.60 USDG of their shares, while the new holder paid nothing. With the budget, the old holders contribute at most what the claims at their write-off needed, which is exactly what the redeem path would take from them.

The keeper can also call `fundClaimsFromWriteOff` best-effort right after `acceptDetachLoss`.

With this change, a write-off gives everyone what redeeming the sink would, even when the sink lost value, and it uses no price and no payment order. The trade-off: until the failed sink pays out, the holders' part and new deposits wait. That follows the vault's existing rule that deposits stay closed while claims are underwater. If the sink never pays, the claims stay owed, as they would on the redeem path.

Tested on a prototype:

- Write-off equals redeem for every holder over 3,001 fuzz runs, with the sink losing 0–90% of its value, plus claims, a later depositor, a liquidation during the detach and partial recovery.
- The PoCs for #21, #23 and #24 stay fixed.
- The #22, #25 and #26 scenarios give exactly the redeem-path result in our differential tests, including two write-offs with a reopen in between.
- A shortfall after a reopen no longer draws on the earlier write-off (0 drawn, the old holders' shares untouched).
- The team's suite passes once `fundClaimsFromWriteOff` is called where claims are expected to be backed, including K25's half-value case. One assertion that expects claims to be paid in shares has to change, because they are now paid in USDG.

REMEDIATION ● FIXED

RAETO

Valid · Low · Fixed (as recommended)

- A write-off no longer prices the failed sink. `sinkRate`, the claims' in-kind part, the claim cuts, `_simCredit` and `_writeOffClaimUnits` are gone.
- Unpaid liquidation claims stay USDG claims. At the booking, `bookingNeedLeft[booking]` records what they need beyond the vault's other USDG.
- New permissionless `fundClaimsFromWriteOff(i)` redeems from the written-off sink only what the claims still need, capped by that budget. If the sink is worth no more than that, it takes everything the sink pays. It falls back to what the sink can pay now (#31). It never takes more, so nothing it brings back can become interest.
- `claimWrittenOff` reverts `ClaimsFirst` while the claims can still be funded from that write-off. The first holder claim releases it, so it can't be drawn for claims again.
- A later shortfall never draws on an earlier write-off: once the claims at its booking are backed, its budget is set to 0.

Regression: `test_K28_WriteOffUsesNoPriceClaimsFirst` follows your scenario (sink at half value): alice and bob end where the redeem path leaves them. `test_K28_LaterShortfallNeverDrawsOnOldWriteOff` covers the budget. The #22, #25 and #26 tests now call `fundClaimsFromWriteOff`, and the claims are paid in USDG.
Fix: PR #34.

L-02 Rounded-down claim cuts at one write-off make a later write-off owe more shares than the vault holds, so the last claimer's part is stuck

SEVERITY LOW STATUS FIXED LOCATION RaetoStockVault.sol

SUMMARY

When a write-off pays part of the liquidation claims in kind, `_setAsideForClaims` lowers the claim pool by the exact value covered. `_simCredit` lowers each claimant's claim by a rounded-down share of it. Afterwards, the claimants' claims add up to a few units more than the pool. At the next write-off, the claimants' part is split by `claim / poolUnits`, so the vault owes slightly more shares of that sink than it holds. The last claimer's `claimWrittenOff` then reverts, and because the holder and claimant parts are paid in one transfer, their whole in-kind part is stuck.

FINDING DESCRIPTION

```
uint256 cutUnits = Math.mulDiv(atBook, w.claimValue, w.poolUnits);   // _simCredit L863: rounded down,
per claimant
...
_liqPoolUnits = units - covered;                                     // _setAsideForClaims L815: exact
total
...
out += Math.mulDiv(w.claimShares, cu, w.poolUnits);                  // _inKind L775 at the next write-
off:
                                                                     // the claims cu add up to more
than poolUnits
```

ATTACK PATH

No attacker is needed.

1. Alice and bob hold liquidation claims. The sink fails; a tenth comes back and the rest is written off, which pays part of the claims in kind.
2. Alice claims her part, so her claim is cut (rounded down). Bob's cut is applied later, the same way.
3. The vault moves to a new sink, which fails and is written off too.
4. Bob claims his part of the second write-off. Alice's `claimWrittenOff(1)` reverts with `ERC20InsufficientBalance`.

IMPACT

In the PoC the vault owes 120.919416e18 shares of the second sink and holds 120.919415e18 (1 wei of USDG short). Alice's part, 90 USDG worth of shares, can't be claimed until someone sends the vault the missing shares. A fuzz run of the same two-write-off flow hit this on its 4th input.

LIKELIHOOD

Low. It needs two write-offs that both pay claims in kind, with at least two claimants whose cuts don't divide evenly.

PROOF OF CONCEPT

`raeto-vaults/test/report/ClaimCutRoundingOverObligatesLaterWriteOff.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {console2} from "forge-std/Test.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {SVLiqBase, MockSink} from "../manual/lib/SVLiqHarness.sol";

/// At a write-off that pays part of the liquidation claims in kind, `_setAsideForClaims` lowers the
claim pool by the
/// exact value covered, while `_simCredit` lowers each claimant's claim by a rounded-DOWN share of it.
Afterwards the
/// claimants' claims add up to more than the pool. At the next write-off the claimants' part is split
by
/// `claim / poolUnits`, so the vault owes more shares of that sink than it holds, and the last
claimer's
/// `claimWrittenOff` reverts: their whole in-kind part is stuck.
/// Harness: the real RaetoStockVault (main at be83889), live Morpho Blue v1.0.0 bytecode (etched), mock
USDG / stock /
/// oracle / sinks. Run (from raeto-vaults/):
///   FOUNDRY_EVM_VERSION=cancun forge test --match-path
test/report/ClaimCutRoundingOverObligatesLaterWriteOff.t.sol -vv
contract ClaimCutRoundingOverObligatesLaterWriteOffTest is SVLiqBase {
    address alice = makeAddr("alice");
    address bob = makeAddr("bob");

    function test_secondWriteOff_owesMoreThanItHolds_lastClaimReverts() public {
        _dep(alice, 10e18);
        _dep(bob, 34354906456380902 24);
        _lever();
        v.accrue();
        _liquidate(_bs() / 3, 4_500); // both hold liquidation claims
        v.accrue();

        // write-off 1: a tenth of the sink comes back, the rest is written off; claims are partly paid
in kind
        vm.prank(owner);
        v.detachSink();
        uint256 all = sink.balanceOf(address(v));
        vm.mockCallRevert(address(sink), abi.encodeWithSelector(MockSink.redeem.selector, all,
address(v), address(v)), "illiquid");
        vm.mockCall(address(sink), abi.encodeWithSelector(IERC4626.maxRedeem.selector, address(v)),
abi.encode(all / 10));
        vm.prank(owner);
        v.redeemRetired(IERC4626(address(sink)));
        vm.clearMockedCalls();
        vm.prank(owner);
        v.acceptDetachLoss(IERC4626(address(sink)));
        vm.prank(alice);
        v.claimWrittenOff(0); // alice is credited (her cut rounded down); bob is not touched yet

        // the vault moves to a new sink, which fails too: write-off 2
        MockSink sink2 = new MockSink(usdg);
        vm.startPrank(owner);
        v.queueSink(IERC4626(address(sink2)));
        vm.warp(block.timestamp + 2 days + 1);
        v.executeSink();
```

```
        v.detachSink();
        v.acceptDetachLoss(IERC4626(address(sink2)));
        vm.stopPrank();

        uint256 owed = v.writtenOffOwed(alice, 1) + v.writtenOffOwed(bob, 1);
        uint256 held = sink2.balanceOf(address(v));
        console2.log("sink2 shares owed to alice + bob / held by the vault", owed, held);
        assertGt(owed, held); //@test the vault owes more than it holds

        vm.prank(bob);
        v.claimWrittenOff(1);
        uint256 aliceOwed = v.writtenOffOwed(alice, 1);
        vm.prank(alice);
        vm.expectRevert(); // ERC20InsufficientBalance
        v.claimWrittenOff(1); //@test the last claimer's whole part is stuck
        console2.log("alice's stuck part (sink2 shares)", aliceOwed);
    }
}
```

Output:

```
Ran 1 test for
test/report/ClaimCutRoundingOverObligatesLaterWriteOff.t.sol:ClaimCutRoundingOverObligatesLaterWriteOffT
est
[PASS] test_secondWriteOff_owesMoreThanItHolds_lastClaimReverts() (gas: 2656003)
Logs:
  sink2 shares owed to alice + bob / held by the vault 120919416000000000000 120919415000000000000
  alice's stuck part (sink2 shares) 9000000000000000000
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.97ms (955.50µs CPU time)
```

RECOMMENDATION

Round each claimant's cut up, so the claims can never add up to more than the pool:

```
uint256 cutUnits = Math.mulDiv(atBook, w.claimValue, w.poolUnits, Math.Rounding.Ceil);
```

Tested:

- The PoC's vault now holds at least what it owes (owed 120.919414e18 vs 120.919415e18 held).
- The two-write-off fuzz passes 2,000 runs (it failed on run 4 before).
- Our write-off property tests still pass.

Rounding up costs each claimant less than one claim unit (1 wei of USDG). An exact running-total split is not possible here, because claimants are credited lazily, one at a time and in any order.

REMEDIATION FIXED

RAETO

Valid · Low · Fixed (by the #28 change)
The claims' in-kind part and the per-claimant claim cuts no longer exist, so there is nothing to round. Each holder's part is a pro-rata share of what is left of the sink, so the last claimer always gets the remainder.
Regression: `test_K30_SecondWriteOffNeverOwesMoreThanHeld` runs your two-write-off flow. Every `claimWrittenOff` goes through, and both sinks end fully handed out.
Fix: PR #34.

L-03 The best-effort withdrawals fall back to `maxWithdraw` / `maxRedeem`, which Morpho VaultV2 always reports as 0, so deleverage and partial recovery take nothing when the sink is short

SEVERITY ● LOW STATUS ● FIXED LOCATION `RaetoStockVault.sol` `RaetoVault.sol`

SUMMARY

The fixes made several withdrawals best-effort: `_withdrawUpTo` (behind `deleverage` and `unwind`), the partial recovery in `redeemRetired`, and `executeSink`'s partial path. When the full amount can't be paid, each falls back to `maxWithdraw` or `maxRedeem`. The sink every vault uses is a Morpho VaultV2 (`0xBeEf…09dd`), whose `maxWithdraw` and `maxRedeem` always return 0, which ERC-4626 allows. On the live sink, `maxWithdraw(RaetoVault)` is 0 while RaetoVault holds 928 USDG of its shares. So when the sink can pay part of what is needed but not all, the vault takes nothing.

FINDING DESCRIPTION

```
// _withdrawUpTo, L1139 (redeemRetired L652 / executeSink L703 / RaetoVault.redeemRetired L406 use maxRedeem)
try s.maxWithdraw(address(this)) returns (uint256 m) {    // Morpho VaultV2: always 0
    if (m > need) m = need;
    if (m > 0) try s.withdraw(m, address(this), address(this)) {} catch {}
} catch {}
```

VaultV2's `withdraw` pays from its idle USDG (16.1M on the live sink) and its liquidity adapter. It never checks `max*`, so a smaller withdrawal would have gone through.

ATTACK PATH

No attacker is needed. This takes a sink liquidity shortfall together with a price move.

1. The sink's payable liquidity drops below what the vault needs.
2. The stock falls, and anyone calls `deleverage()`. The full withdrawal reverts, the fallback reads 0, and nothing is repaid.
3. If the fall continues past the liquidation threshold, Morpho liquidates the vault's collateral, with the liquidation bonus.

IMPACT

Measured in the PoC, with the sink able to pay 100 of the vault's 900:

	SINK THAT REPORTS MAX*	VAULTV2 BEHAVIOUR (MAX* = 0)
Debt after `deleverage`	900 → 800	900 → **900**
Recovered in part by `redeemRetired`	100	**0** (exits stay paused)

LIKELIHOOD

Low. It needs the sink to be able to pay some, but not all, of what the vault needs. The vault's needs are small against VaultV2's 16.1M idle, so that band is narrow.

PROOF OF CONCEPT

`raeto-vaults/test/report/BestEffortWithdrawDeadOnVaultV2.t.sol`:

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {console2} from "forge-std/Test.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
import {SVLiqBase, MockToken} from "../manual/lib/SVLiqHarness.sol";
import {RaetoStockVault} from "../../src/RaetoStockVault.sol";

/// A lending vault that pays withdrawals from its idle USDG only (the rest is lent out), like Morpho VaultV2 without
/// its liquidity adapter. With `reportsMax` false its maxWithdraw / maxRedeem return 0, as VaultV2's do
/// (VaultV2.sol 744-751); with true they report what it can pay.
contract IdleOnlySink is ERC20 {
    using Math for uint256;

    MockToken public immutable usd;
    bool public immutable reportsMax;
    uint256 public lent;

    constructor(MockToken u, bool reports) ERC20("idle-only sink", "IOS") {
        usd = u;
        reportsMax = reports;
    }

    function asset() external view returns (address) {
        return address(usd);
    }

    function totalAssets() public view returns (uint256) {
        return usd.balanceOf(address(this)) + lent;
    }

    function convertToAssets(uint256 s) public view returns (uint256) {
        return s.mulDiv(totalAssets() + 1, totalSupply() + 1e12);
    }

    function deposit(uint256 a, address to) external returns (uint256 s) {
        s = a.mulDiv(totalSupply() + 1e12, totalAssets() + 1);
        usd.transferFrom(msg.sender, address(this), a);
        _mint(to, s);
    }

    function withdraw(uint256 a, address to, address o) external returns (uint256 s) {
        s = a.mulDiv(totalSupply() + 1e12, totalAssets() + 1, Math.Rounding.Ceil);
        _burn(o, s);
        usd.transfer(to, a); // reverts if idle < a
    }

    function redeem(uint256 s, address to, address o) external returns (uint256 a) {
        a = convertToAssets(s);
        _burn(o, s);
        usd.transfer(to, a);
    }
```

```
    function maxWithdraw(address o) external view returns (uint256) {
        if (!reportsMax) return 0;
        uint256 v = convertToAssets(balanceOf(o));
        uint256 idle = usd.balanceOf(address(this));
        return v < idle ? v : idle;
    }

    function maxRedeem(address o) external view returns (uint256) {
        if (!reportsMax) return 0;
        uint256 idle = usd.balanceOf(address(this));
        uint256 b = balanceOf(o);
        return convertToAssets(b) ⩽ idle ? b : idle.mulDiv(totalSupply() + 1e12, totalAssets() + 1);
    }

    /// The market borrows `a`: it leaves the vault's idle.
    function lendOut(uint256 a) external {
        usd.transfer(address(0xdead), a);
        lent += a;
    }
}

/// Every best-effort withdrawal added by the fixes (`_withdrawUpTo` for deleverage/unwind, the partial
paths of
/// `redeemRetired` and `executeSink`) falls back to `maxWithdraw` / `maxRedeem` when the full amount
can't be paid.
/// Morpho VaultV2, the sink every vault uses, returns 0 from both, so whenever the sink can pay part of
what is
/// needed but not all, the vault takes nothing.
/// Harness: the real RaetoStockVault (main at be83889), live Morpho Blue v1.0.0 bytecode (etched), mock
USDG / stock /
/// oracle and the sink above. Run (from raeto-vaults/):
///   FOUNDRY_EVM_VERSION=cancun forge test --match-path
test/report/BestEffortWithdrawDeadOnVaultV2.t.sol -vv
contract BestEffortWithdrawDeadOnVaultV2Test is SVLiqBase {
    address alice = makeAddr("alice");

    function _vaultOn(IdleOnlySink s) internal returns (RaetoStockVault vv) {
        vv = new RaetoStockVault(morpho, mp, IERC4626(address(s)), treasury, keeper, owner, 1e24, 1e15,
10);
        usdg.mint(owner, 1e6);
        vm.startPrank(owner);
        usdg.approve(address(vv), 1e6);
        vv.fundReserve(1e6);
        vm.stopPrank();
        stock.mint(alice, 10e18);
        vm.startPrank(alice);
        stock.approve(address(vv), 10e18);
        vv.deposit(10e18);
        vm.stopPrank();
        vm.prank(keeper);
        vv.rebalance(); // borrows 900 USDG into the sink
    }

    function test_deleverage_repaysNothing_whenSinkCanPayPart() public {
        uint256[2] memory after_;
        for (uint256 k; k < 2; k++) {
            IdleOnlySink s = new IdleOnlySink(usdg, k == 0);
```

```
            RaetoStockVault vv = _vaultOn(s);
            s.lendOut(800e6); // 100 of the vault's 900 is still idle in the sink
            oracle.setPrice(P0 * 70 / 100); // LTV 43%: above maxLtv, deleverage should repay toward 30%
            vv.deleverage();
            after_[k] = vv.debt();
            oracle.setPrice(P0);
        }
        console2.log("debt 900 → after deleverage: sink reports max / VaultV2 (max = 0)", after_[0],
after_[1]);
        assertLt(after_[0], 900e6);
        assertEq(after_[1], 900e6); //@test nothing repaid, although the sink could pay 100
    }

    function test_redeemRetired_recoversNothingInParts() public {
        uint256[2] memory got;
        for (uint256 k; k < 2; k++) {
            IdleOnlySink s = new IdleOnlySink(usdg, k == 0);
            RaetoStockVault vv = _vaultOn(s);
            s.lendOut(800e6);
            vm.prank(owner);
            vv.detachSink();
            uint256 b0 = usdg.balanceOf(address(vv));
            vm.prank(owner);
            vv.redeemRetired(IERC4626(address(s)));
            got[k] = usdg.balanceOf(address(vv)) - b0;
            assertTrue(vv.detachPending());
        }
        console2.log("recovered in part: sink reports max / VaultV2 (max = 0)", got[0], got[1]);
        assertGt(got[0], 0);
        assertEq(got[1], 0); //@test recovery in parts takes nothing; exits stay paused
    }
}
```

Output:

```
Ran 2 tests for test/report/BestEffortWithdrawDeadOnVaultV2.t.sol:BestEffortWithdrawDeadOnVaultV2Test
[PASS] test_deleverage_repaysNothing_whenSinkCanPayPart() (gas: 16234509)
Logs:
  debt 900 → after deleverage: sink reports max / VaultV2 (max = 0) 800000000 900000000
[PASS] test_redeemRetired_recoversNothingInParts() (gas: 16160525)
Logs:
  recovered in part: sink reports max / VaultV2 (max = 0) 100000000 0
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.97ms (2.96ms CPU time)
```

RECOMMENDATION

Don't size the fallback from `max*` alone. When it reports 0, fall back to the USDG the sink holds idle, which VaultV2 pays out first, capped at the vault's own position:

```
function _payableNow(IERC4626 s) internal view returns (uint256 m) {
    try s.maxWithdraw(address(this)) returns (uint256 x) { m = x; } catch {}
    if (m == 0) m = usdg.balanceOf(address(s)); // VaultV2 reports 0; its idle USDG is what it pays
first
    try s.convertToAssets(s.balanceOf(address(this))) returns (uint256 mine) { if (m > mine) m = mine; }
catch {}
}

// _withdrawUpTo:                   uint256 m = _payableNow(s); if (m > need) m = need; try
s.withdraw(m, ...)
// redeemRetired / executeSink:     uint256 m = _payableNow(old); try old.withdraw(m, ...)
```

The fallback only runs after the full withdrawal has failed. At that point the vault's position is worth more than the sink can pay, so `m` is always within the vault's own shares.

For a sink other than VaultV2, the idle balance is only a hint. The withdrawal is still attempted in a `try`, and every caller measures what actually arrived. Add a fork test against the live VaultV2 as well as a mock whose `max*` returns 0.

Tested: with VaultV2's behaviour, `deleverage` repays 100 (debt 900 → 800) and partial recovery takes 100. Our write-off property tests and the team's suite pass. Add regression tests with a sink whose `max*` return 0.

REMEDIATION ● FIXED

RAETO

Valid · Low · Fixed (as recommended)

- New `_payableNow(s)` : it uses `maxWithdraw`, and when that reports 0 it falls back to the sink's idle USDG, capped at the vault's own position.
- `_withdrawUpTo` (deleverage, unwind), the partial paths of `redeemRetired` and `executeSink`, and RaetoVault's `redeemRetired` now try `withdraw(m)` with that amount instead of `maxRedeem`. Every caller still measures what actually arrived.
- Confirmed on the live sink: `maxWithdraw` on 0xBeEf...09dd is 0 for our vaults while they hold its shares.

Regression: `ZeroMaxSink` (max* = 0, pays only from idle, like VaultV2). In `test_K31_BestEffortWorksWhenMaxReportsZero`, deleverage repays the 100 the sink can pay and partial recovery takes 100. `test_K31_PartialRecoveryWhenMaxReportsZero` covers RaetoVault. Both fail without the fix.
Fix: PR #34.

L-04 The `dustTolerance` waiver is checked per withdrawal, so a holder can split an in-loss exit and leave their gap to the holders who stay

SEVERITY LOW STATUS FIXED LOCATION `RaetoStockVault.sol`

SUMMARY

When the vault is in loss, a withdrawer adds their share of the gap, "so leaving first gains nothing and the holders who stay are unaffected" (L29-31). `withdrawTo` waives the gap when it is within `dustTolerance`, and it checks that per call, on a size the caller picks. A holder can split one exit into many withdrawals whose gap is each within the tolerance, and pay almost nothing. The vault repays each part of the loan from shared USDG, so the holders who stay pay the waived gap.

The deployed vaults use a tolerance of 10 wei, where this costs more in gas than it saves. `dustTolerance` is an immutable that is chosen again at the redeploy.

FINDING DESCRIPTION

```
uint256 owe = _toAssetsUp(repayShares);
uint256 mine = _depositorUsdgFor(shares, ts);
if (owe > mine + dustTolerance) {          // L411: per call, on a size the caller chooses
    ... // charge the gap
}
_repayShares(repayShares);                 // the vault repays `owe` from shared USDG either way
```

ATTACK PATH

1. The vault is in loss, for example because its sink lost value.
2. A holder repeatedly withdraws the largest amount whose `shortfallFor` is 0, with `maxShortfallIn = 0`.
3. Their total gap is about 0. The holders who stay pay it instead.

IMPACT

Measured with a 1 USDG tolerance, where alice's share of the gap is 90:

	ALICE PAYS	BOB (STAYS) PAYS
Alice exits in one go	90.00	90.00
Alice splits her exit into 125 withdrawals	**0.00**	**180.00**

LIKELIHOOD

Low, and latent. It needs the vault in loss and a tolerance large relative to gas. At the deployed 10 wei it is uneconomic.

PROOF OF CONCEPT

`raeto-vaults/test/report/SplitExitDodgesShortfall.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {console2} from "forge-std/Test.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {SVLiqBase} from "../manual/lib/SVLiqHarness.sol";
import {RaetoStockVault} from "../../src/RaetoStockVault.sol";

/// `withdrawTo` waives a withdrawer's share of the in-loss gap whenever it is within `dustTolerance`,
and checks that
/// per call, on a size the caller picks. A holder of a vault in loss can split one exit into many
withdrawals whose
/// gap is each within the tolerance and pay almost nothing; the holders who stay pay the rest. The
deployed vaults use
/// a tolerance of 10 wei (uneconomic); `dustTolerance` is an immutable that is chosen again at the
redeploy.
/// Harness: the real RaetoStockVault (main at be83889) with dustTolerance = 1 USDG, live Morpho Blue
v1.0.0 bytecode
/// (etched), mock USDG / stock / oracle / sink. Run (from raeto-vaults/):
///   FOUNDRY_EVM_VERSION=cancun forge test --match-path test/report/SplitExitDodgesShortfall.t.sol -vv
contract SplitExitDodgesShortfallTest is SVLiqBase {
    address alice = makeAddr("alice");
    address bob = makeAddr("bob");
    RaetoStockVault vv;

    function _dep2(address u, uint256 amt) internal {
        stock.mint(u, amt);
        vm.startPrank(u);
        stock.approve(address(vv), amt);
        vv.deposit(amt);
        usdg.approve(address(vv), type(uint256).max);
        vm.stopPrank();
        usdg.mint(u, 10_000e6);
    }

    /// The largest withdrawal whose gap the vault waives right now.
    function _largestWaived(uint256 max) internal view returns (uint256 lo) {
        uint256 hi = max;
        while (lo < hi) {
            uint256 mid = (lo + hi + 1) / 2;
            if (vv.shortfallFor(mid) == 0) lo = mid;
            else hi = mid - 1;
        }
    }

    function test_splitExit_paysNoGap_holderWhoStaysPaysIt() public {
        vv = new RaetoStockVault(morpho, mp, IERC4626(address(sink)), treasury, keeper, owner, 1e24,
1e15, 1e6);
        usdg.mint(owner, 1e6);
        vm.startPrank(owner);
        usdg.approve(address(vv), 1e6);
        vv.fundReserve(1e6);
        vm.stopPrank();
        _dep2(alice, 10e18);
        _dep2(bob, 10e18);
        vm.prank(keeper);
```

```
        vv.rebalance();
        sink.slash(sink.totalAssets() / 10); // the sink loses 10%: the vault is in loss
        vv.accrue();
        uint256 snap = vm.snapshotState();

        // alice leaves in one go, then bob
        uint256 a0 = usdg.balanceOf(alice);
        vm.prank(alice);
        vv.withdrawTo(10e18, alice, 0, type(uint256).max);
        uint256 aliceOneShot = a0 - usdg.balanceOf(alice);
        uint256 b0 = usdg.balanceOf(bob);
        vm.prank(bob);
        vv.withdrawTo(10e18, bob, 0, type(uint256).max);
        uint256 bobAfterOneShot = b0 - usdg.balanceOf(bob);
        vm.revertToState(snap);

        // alice splits her exit into withdrawals that each stay within the tolerance (maxShortfallIn =
0)
        a0 = usdg.balanceOf(alice);
        uint256 calls;
        while (vv.sharesOf(alice) > 0) {
            uint256 s = _largestWaived(vv.sharesOf(alice));
            if (s == 0) break;
            vm.prank(alice);
            vv.withdrawTo(s, alice, 0, 0);
            calls++;
        }
        uint256 aliceSplit = a0 - usdg.balanceOf(alice);
        uint256 left = vv.sharesOf(alice);
        if (left > 0) {
            vm.prank(alice);
            vv.withdrawTo(left, alice, 0, type(uint256).max);
            aliceSplit = a0 - usdg.balanceOf(alice);
        }
        b0 = usdg.balanceOf(bob);
        vm.prank(bob);
        vv.withdrawTo(10e18, bob, 0, type(uint256).max);
        uint256 bobAfterSplit = b0 - usdg.balanceOf(bob);

        console2.log("alice's gap: one exit / split exit", aliceOneShot, aliceSplit);
        console2.log("withdrawals in the split exit", calls);
        console2.log("bob's gap after alice: one exit / split exit", bobAfterOneShot, bobAfterSplit);
        assertLt(aliceSplit, aliceOneShot / 10); //@test the split exit pays almost none of alice's gap
        assertApproxEqAbs(bobAfterSplit - bobAfterOneShot, aliceOneShot - aliceSplit, 10); //@test bob
pays it instead
    }
}
```

Output:

```
Ran 1 test for test/report/SplitExitDodgesShortfall.t.sol:SplitExitDodgesShortfallTest
[PASS] test_splitExit_paysNoGap_holderWhoStaysPaysIt() (gas: 148332758)
Logs:
  alice's gap: one exit / split exit 90000000 0
  withdrawals in the split exit 125
  bob's gap after alice: one exit / split exit 90000000 180000001
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 210.02ms (208.78ms CPU time)
```

RECOMMENDATION

Waive a rounding-size gap only while the vault is *not* in loss. When it is in loss, always charge the gap. Healthy exits then still need no USDG approval, and splitting an in-loss exit gains nothing:

```
if (owe > mine) {
    uint256 gap = owe - mine;
    if (gap > dustTolerance || inLoss()) { // a healthy vault waives rounding; a vault in loss always charges
        if (gap > maxShortfallIn) revert InLoss();
        usdg.safeTransferFrom(msg.sender, address(this), gap);
        emit ShortfallPaid(msg.sender, gap);
    }
}
// shortfallFor: return gap > dustTolerance || inLoss() ? gap : 0
```

`inLoss()` already allows `dustTolerance`. If the vault's whole deficit is within it, splitting can shift at most that deficit, never `dustTolerance` per call.

We first tried paying rounding-size gaps from `reserve`, but a split exit could then drain the reserve. That blocked the keeper's borrowing (`rebalance` reverts with `NoReserve`) until someone refilled it. The gate above leaves the reserve untouched.

Tested:

- The split exit now pays alice's full 90, and bob pays exactly 90 either way.
- The reserve is untouched, and `rebalance` still works.
- The team's suite passes.

REMEDIATION ● FIXED

RAETO

Valid · Low · Fixed (as recommended)
`withdrawTo` and `shortfallFor` waive a rounding-size gap only while the vault is not in loss: `if (gap > dustTolerance || inLoss())`. The reserve is untouched.
Regression: `test_K32_SplitExitPaysTheWholeGap` uses `dustTolerance` = 1 USDG and splits an in-loss exit into 100 slices, each under the tolerance. Each slice is charged, and the total equals the one-shot gap. It fails without the fix.
Fix: PR #34.

I-01 The stock vault counts empty write-offs toward `MAX_WRITE_OFFS`

SEVERITY ● INFORMATIONAL STATUS ● FIXED LOCATION `RaetoStockVault.sol`

- **What happens:** `acceptDetachLoss` (L734) checks the cap before it pushes any write-off, including one that holds nothing. RaetoVault's `_writeOff` skips those first.
- **Effect:** in our test, four write-offs of empty sinks used up the cap, so the 5th `acceptDetachLoss` reverts with `TooManyWriteOffs` and exits stay paused, as in L-02.

Fix: record a write-off only if the vault holds shares of the sink, or can't read its balance. An empty one is still marked written off, but it isn't pushed and doesn't use the cap. With L-01's fix there is no per-claimant loop left, so the cap can also be raised. Tested: five empty write-offs no longer block the next one.

REMEDIATION ● FIXED

RAETO

I-01 · Fixed. An empty sink (balance read as 0) is marked written off but not pushed, so it doesn't use the cap. A sink that can't be read is still recorded. `MAX_WRITE_OFFS` is now 8. Test: `test_K33_EmptyWriteOffsDontUseTheCap` (six empty write-offs).

I-02 A liquidation that repays every borrow share skips the pre-book

SEVERITY INFORMATIONAL STATUS FIXED LOCATION RaetoStockVault.sol

- **What happens:** `_bookLiquidation` books the equity change before a liquidation only `if (bs > 0)` (L989). The aim is to avoid valuing a loan that Morpho wrote off as bad debt. But a liquidator who repays every borrow share also reaches `bs == 0`, with collateral still left.
- **Effect:** any yield or loss since the last `accrue` then goes into the holders' liquidation claims instead of Raeto's pending yield. In our test, 89.20 USDG of unbooked sink yield went to the claimant: pending 0 and claim 989.20, against 89.20 and 900 when the yield was booked first. The same branch also lets a tiny seizure plus a gift repay move unbooked yield into claims.
- **Bound:** limited to the change since the last `accrue`.

Fix: Morpho writes off bad debt, and resets the borrow share price, only when no collateral is left. So pre-book, and apply the `repaid` cap, whenever collateral remains:

```
bool valued = bs > 0 || live > 0;   // pass `live` from _accrue
if (valued) { /* pre-book against _toAssetsUp(exp) */ }
...
if (valued) { /* cap freed at _toAssetsDown(exp - bs) */ }
```

Tested: the full-share liquidation now books 89.20 to pending yield and 900 to the claim, the same as booking first. The tiny-seizure-plus-gift skim is gone (the holder nets 0). The full wipe with bad debt is unchanged, and our property tests and the team's suite pass.

REMEDIATION FIXED

RAETO

I-02 · Fixed (as recommended). `_bookLiquidation(exp, bs, live)` pre-books, and caps what is freed at the repaid amount, whenever `bs > 0 || live > 0`. Test: `test_K33_FullShareLiquidationStillPreBooks` checks that a liquidation repaying every borrow share with collateral left gives the same pending yield and claim as booking first.

I-03 RaetoVault can re-attach a source that is still pending; writing it off then counts its shares twice

SEVERITY INFORMATIONAL STATUS FIXED LOCATION RaetoVault.sol

- **What happens:** `executeSource` (L327) doesn't check `detachPending()`; RaetoStockVault's `executeSink` does. If the owner re-attaches a detached source before resolving it, then calls `acceptDetachLoss` on it (L424), the *active* source is written off.
- **Effect:** its shares are handed out in kind while `totalAssets()` still counts them and `_pay` still spends them. With two depositors of 1,000 each, the first out leaves with 2,000 and the second with 0.
- **Why Info:** it needs two unusual actions by the trusted owner.

Fix: in `executeSource`, add `if (detachPending()) revert Paused();` and set `retiredSource[next] = false`. In `acceptDetachLoss`, add `if (address(old) == address(source)) revert SameSource();`. I-04's fix also refuses to queue a pending source.

REMEDIATION FIXED

RAETO

I-03 · Fixed. In RaetoVault, `executeSource` reverts `Paused` while a detach is pending and sets `retiredSource[next] = false`. `acceptDetachLoss` reverts `SameSource` on the active source.

I-04 A detach freezes exits through a queued venue switch, which voids the 2-day exit window

SEVERITY INFORMATIONAL STATUS FIXED LOCATION `RaetoVault.sol` `RaetoStockVault.sol`

- **What happens:** the 2-day `SOURCE_DELAY` / `SINK_DELAY` exists "so users can exit first" (RaetoVault L309). But `detachSource` / `detachSink` pause every exit at once, neither cancels a queued switch, and queuing doesn't check for a pending detach. The owner can queue and detach in the same block. Exits stay frozen for the whole delay, then one batch (`redeemRetired` , then `executeSource` / `executeSink`) completes the switch.
- **Effect:** RaetoVault moved all 1,000 of principal into the new source, and the stock vault borrowed 900 into the new sink, while every exit reverted `Paused` during the delay. The live owner of every vault is a single EOA with an EIP-7702 delegation.

Fix: in `queueSource` / `queueSink` , add `if (detachPending()) revert Paused();` . In `detachSource` / `detachSink` , clear the queued source/sink and its eta. Then every switch starts its full delay with exits open. A multisig owner would also limit what a single key can do.

REMEDIATION FIXED

RAETO

I-04 · Fixed. `queueSource` / `queueSink` revert `Paused` while a detach is pending, and `detachSource` / `detachSink` clear the queued venue and its eta. Tests: `test_K33_DetachCancelsQueuedSwitch` and `test_K33_NoSwitchWhileDetachPending` .

09 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

10 - WHO WE ARE

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.

EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.

AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.

FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.

INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.

KANN AUDITS

WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits

© 2026 Kann Audits. Prepared for Raeto.